

1.1. Boolean Algebra

Boolean algebra is a branch of Algebra (Mathematics) that deals with operations on logical values with Boolean variables, Boolean variables are represented as binary numbers which takes logic 1 and logic 0 values. Hence, the Boolean algebra is also called two-valued logic, Binary Algebra or Logical Algebra. The Boolean algebra was introduced by great mathematician George Boole in 1847. The Boolean algebra is a fundamental for the development of digital electronic systems, and is provided for in all programming languages. Set theory and statistics fields also uses Boolean algebra for the representation, simplification and analysis of mathematical quantities.

Logic levels are classified into two types

1. Positive logic

Logic 0 = False, 0V, Open Switch, OFF

Logic 1= True, +5V, Closed Switch, ON

2. Negative logic

Logic 0 = True, +5V, Closed Switch, ON

Logic 1= False, 0V, Open Switch, OFF

Boolean algebra differs from normal or elementary algebra. Latter deals with numerical operations such as, addition, subtraction, multiplication and division on decimal numbers. And former deals with the logical operations such as conjunction (OR), disjunction(AND) and negation(NOT).

In present context, positive logic has been used for the entire discussion, representation and simplification of Boolean variables.

1.1.1. Rules and properties of Boolean Algebra

1. Boolean variables takes only two values, logic 1 and logic 0, called binary numbers.
2. Basic operations of Boolean algebra are complement of a variable, ORing and ANDing of two or more variables.
3. Mathematical description of Boolean operations using variables is called Boolean expression.
4. Complement of variable is represented by an over-bar (-).
Example: $Y = \overline{A}$, Y is the output variable
5. ORing of variables is represented by a plus symbol (+)
Example: $Y = A + B$, Y is the output variable
6. ANDing of variables is represented by a dot symbol (.)
Example: $Y = A . B$, Y is the output variable
7. Boolean operations are different from binary operations.
*Example : $1+1=10$ in Binary Addition
 $1+1=1$ in Boolean algebra.*

Table 1.1, shows the complement operation of a variable, table 1.2 summarized the OR operation and table 1.3, summarized the AND operation of two variables.

A	$Y = \overline{A}$
0	1
1	0

Table 1.1: Complement of variable A

A	B	Y=A+B
0	0	0
0	1	1
1	0	1
1	1	1

Table 1.2: OR operation on A and B

A	B	Y=A.B
0	0	0
0	1	0
1	0	0
1	1	1

Table 1.3: AND operation on A and B

The present chapter deals with the simplification of Boolean expressions and representation using sum of product form and product of sum forms.

1.1.2. Boolean Laws:

Law-1: Commutative law

The sequence of changing the variables does not effect on the result even after changing their sequence while performing OR/AND operations of Boolean expression.

$$\text{i. e. , } A \cdot B = B \cdot A \text{ and } A + B = B + A$$

Law-2: Associative law

The order of operations on variables is independent.

$$A \cdot (B \cdot C) = (A \cdot B) \cdot C \text{ and } A + (B + C) = (A + B) + C$$

Law-3: Distributive Laws

$$A \cdot (B + C) = A \cdot B + A \cdot C$$

$$A + BC = (A + B)(A + C)$$

Law-4: AND Laws

$$A \cdot 0 = 0$$

$$A \cdot 1 = A$$

$$A \cdot A = A$$

$$A \cdot \bar{A} = 0$$

Law-5: OR Laws

$$A + 0 = A$$

$$A + 1 = 1$$

$$A + A = A$$

$$A + \bar{A} = 1$$

Law-6: Inversion/Complement/NOT Laws

$$0 = 1$$

$$1 = 0$$

$$\bar{\bar{A}} = A$$

Law-7: Absorption Law

$$\begin{aligned} A(A + B) &= A \\ A + AB &= A \\ A + \bar{A}B &= A + B \end{aligned}$$

Law-8: Demargon's Laws

De-Morgan's First Law

Statement: Sum of complement of two or more variables is equal to the product of the complement of their variables.

$$\text{i. e., } \bar{\bar{A}} + \bar{\bar{B}} + \bar{\bar{C}} + \dots = \bar{A} \cdot \bar{B} \cdot \bar{C} \dots$$

Proof:

consider three variables for the proof shown in figure 1.4

A	B	C	$\bar{\bar{A}} + \bar{\bar{B}} + \bar{\bar{C}}$	$\bar{A} \cdot \bar{B} \cdot \bar{C}$
0	0	0	1	1
0	0	1	0	0
0	1	0	0	0
0	1	1	0	0
1	0	0	0	0
1	0	1	0	0
1	1	0	0	0
1	1	1	0	0

Table 1.4: De-Morgan's First Law

De-Morgan's Second Law

Statement: Product of complement of two or more variables is equal to the sum of the complement of their variables.

$$\text{i. e., } \bar{\bar{A}}\bar{\bar{B}}\bar{\bar{C}} + \dots = \bar{A} + \bar{B} + \bar{C} \dots$$

Proof:

consider three variables for the proof shown in figure 1.5

A	B	C	$\bar{\bar{A}}\bar{\bar{B}}\bar{\bar{C}}$	$\bar{A} + \bar{B} + \bar{C}$
0	0	0	1	1
0	0	1	1	1
0	1	0	1	1
0	1	1	1	1
1	0	0	1	1
1	0	1	1	1
1	1	0	1	1
1	1	1	0	0

Table 1.5: De-Morgan's Second Law

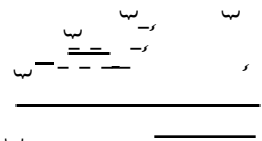
Boolean expressions must be simplified and evaluated using the order of operator precedence shown in table 1.6

Operator	Precedence
Parenthesis	1
NOT	2
AND	3
OR	4

Table
1.6:
Operator
precedence

Example:

$$Y = (A + (C + \bar{D}) + \bar{B} - \bar{C}) \bar{E}$$



1.1.3. Simplify the following expressions

1. $Y = BC + \bar{B}C + BA$

solution:

$$Y = BC + \bar{B}C + BA$$

$$Y = B(C + \bar{C}) + BA$$

$$Y = B + BA \quad (\because C + \bar{C} = 1)$$

$$Y = B(1 + A) \quad (\because 1 + A = 1)$$

$$= B \cdot 1$$

$$Y = B$$

2. $Y = A + \bar{A}B + \bar{A}\bar{B}C + \bar{A}\bar{B}\bar{C}D + \bar{A}\bar{B}\bar{C}\bar{D}E$

Solution:

$$Y = A + \bar{A}B + \bar{A}\bar{B}C + \bar{A}\bar{B}\bar{C}D + \bar{A}\bar{B}\bar{C}\bar{D}E$$

$$Y = A + \bar{A}(B + \bar{B}C + \bar{B}\bar{C}D + \bar{B}\bar{C}\bar{D}E)$$

$$Y = A + (B + \bar{B}C + \bar{B}\bar{C}D + \bar{B}\bar{C}\bar{D}E) \quad (\because A + \bar{A}B = A + B)$$

$$Y = A + (B + \bar{B}(C + \bar{C}D + \bar{C}\bar{D}E)) \quad (\because A + \bar{A}B = A + B)$$

$$Y = A + (B + (C + \bar{C}D + \bar{C}\bar{D}E)) \quad (\because A + \bar{A}B = A + B)$$

$$Y = A + (B + (C + \bar{C}(D + \bar{D}E))) \quad (\because A + \bar{A}B = A + B)$$

$$Y = A + (B + (C + (D + \bar{D}E))) \quad (\because A + \bar{A}B = A + B)$$

$$Y = A + B + C + D + E$$

3. $Y = C + \bar{B}\bar{C}$

Solution:

$$Y = C + \bar{B} + \bar{C}$$

$$Y = (C + \bar{C}) + \bar{B}$$

$$Y = 1 + \bar{B}$$

$$Y = 1$$

4. $Y = \bar{A}\bar{B}(A + B)(B + B)$

Solution:

$$Y = (\bar{A} + \bar{B})(\bar{A} + B)(1)$$

$$Y = \bar{A}\bar{A} + \bar{A}B + \bar{B}\bar{A} + \bar{B}B$$

$$Y = 0 + \bar{A}(B + \bar{B}) + 0$$

$$Y = \bar{A}(1)$$

$$Y = \bar{A}$$

5. $Y = (A + C)(AD + \bar{A}D) + AC + C$

Solution:

$$Y = (A + C)(A(D + \bar{D})) + AC + C$$

$$Y = (A + C)(A)(1) + AC + C$$

$$Y = AA + AC + AC + C$$

$$Y = A + AC + C$$

$$Y = A(1 + C) + C$$

$$Y = A + C$$

$$6. Y = A(A + B) + (B + AA)(A + B)$$

Solution:

$$Y = \bar{A}A + \bar{A}B + BA + B\bar{B} + AAA + AA\bar{B}$$

$$Y = 0 + \bar{A}B + AB + 0 + A + A\bar{B}$$

$$Y = B(\bar{A} + A) + A + A\bar{B}$$

$$Y = B + A(1 + \bar{B})$$

$$Y = A + B$$

$$7. Y = \bar{A} + BC + D(E + F)$$

Solution:

$$Y = (A + BC)(D + E + F)$$

$$(\quad) -$$

$$Y = A + BC (D + (E + F)$$

$$Y = (A + BC)(D + E + F)$$

$$8. Y = AB + A(B + C) + B(B + C)$$

Solution:

$$Y = AB + AB + AC + BB + BC$$

$$Y = AB + AC + B + BC$$

$$Y = AB + AC + B(1 + C)$$

$$Y = AB + AC + B$$

$$Y = B(A + 1) + AC$$

$$Y = B + AC$$

$$9. Y = AB + A(\bar{B} + \bar{C}) + B(\bar{B} + \bar{C})$$

Solution:

$$Y = A\bar{B} + A(\bar{B}\bar{C}) + B\bar{B}\bar{C}$$

$$Y = A\bar{B}(1 + \bar{C})$$

$$Y = A\bar{B}$$

$$10. Y = ABC + ABC + ABC + ABC + ABC$$

Solution:

$$Y = \bar{A}BC + A\bar{B}\bar{C} + \bar{A}\bar{B}\bar{C} + A\bar{B}C + ABC$$

$$Y = \bar{A}BC + A\bar{B}\bar{C} + \bar{A}\bar{B}\bar{C} + AC(\bar{B} + B)$$

$$Y = \bar{A}BC + A\bar{B}\bar{C} + \bar{A}\bar{B}\bar{C} + AC$$

$$Y = \bar{A}BC + \bar{B}\bar{C}(A + \bar{A}) + AC$$

$$Y = \bar{A}BC + \bar{B}\bar{C} + AC$$

$$Y = (\bar{A}B + A)C + \bar{B}\bar{C}$$

$$Y = (A + B)C + \bar{B}\bar{C}$$

$$Y = AC + BC + \bar{B}\bar{C}$$

11. If $F = A + B$, $A = Y + X$ and $B = X + Y$, then $F = ?$

Solution:

$$F = (\bar{Y} + \bar{X}) + (\bar{X} + Y)$$

$$F = \bar{Y} \cdot \bar{X} + \bar{X} + Y$$

$$\mathbf{F = Y + X}$$

12. If $f(A, B) = A + B$, then show that $f(f(X, YZ), XY) = X + Y$

Solution:

$$f(A, B) = f(X, YZ)$$

$$\therefore A = X \text{ and } B = YZ$$

$$f(X, YZ) = X + YZ$$

$$f(f(X, YZ), \bar{X}Y) = f(A + B)$$

$$A = f(X, YZ) \text{ and } B = \bar{X}Y$$

$$f(f(X, YZ), \bar{X}Y) = (X + YZ) + \bar{X}Y$$

$$f(f(X, YZ), \bar{X}Y) = (X + \bar{X}Y) + YZ$$

$$f(f(X, YZ), \bar{X}Y) = (X + Y) + YZ$$

$$f(f(X, YZ), \bar{X}Y) = (X + Y(1 + Z))$$

$$\mathbf{f(f(X, YZ), XY) = (X + Y)}$$

13. Let $A * B = A + B$ and $C = A * B$ then $C * A$ is _____

Solution:

$$C * A = \bar{C} + A$$

$$C * A = (\bar{A} * \bar{B}) + A$$

$$C * A = (\bar{A} + \bar{B}) + A$$

$$C * A = A \cdot \bar{B} + A$$

$$\mathbf{C * A = A}$$

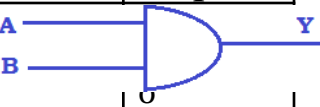
1.2. Realization of Boolean expressions using logic gates.

Logic gate is the basic building block of any digital circuits. The logic gates may have one or more inputs and only one output. The relationship between input and output is based on a certain logic, which is same as Boolean operations, such as AND, OR and NOT.

Based on the Boolean operations, the gates are named as AND gate, OR gate and NOT gate. These three gates are called basic gates, and some more gates can be derived by using the basic gates, they are named as NAND gate, NOR gate, EXOR gate and XNOR gate. NAND and NOR gates are called universal gates, because by using only the NAND gates /NOR gates we can realize all basic gates even all Boolean expression.

Logic gates, its truth table, expression and symbols are summarized in the table 1.7 as follows.

Sl. No. Gate name and Logic Symbol Truth table and Logical Expression

Inputs		Output
A	B	
0	0	
0	1	
1	0	
1	1	

AND Gate

1

Inputs		Output
A	B	$Y = A + B$

2 OR Gate



0	0	0
0	1	1
1	0	1
1	1	1

Inputs	Output
A	$Y = \bar{A}$
0	1
1	0

NOT Gate

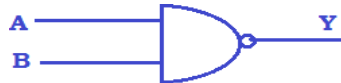
3



Inputs		Output
A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

NAND Gate

4



Inputs		Output
A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

NOR Gate

5



Inputs		Output
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

EX-OR Gate

6



Inputs		Output
A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

EX-NOR Gate

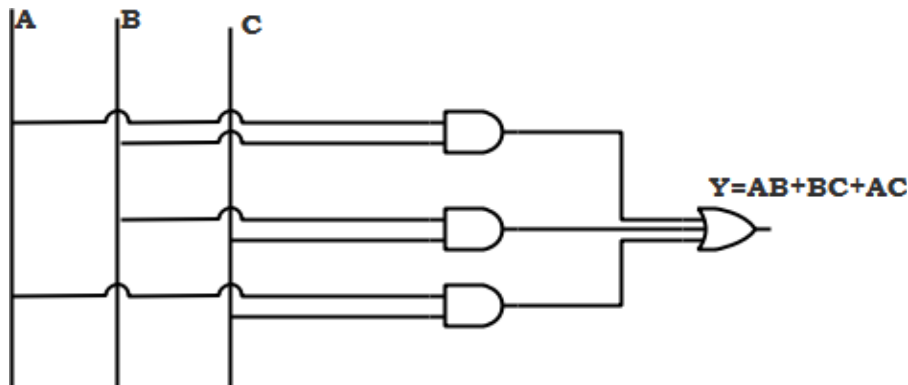
7



1.2.1. Realize the following Boolean expression using basic gates.

$$Y = AB + BC + AC$$

Logic diagram

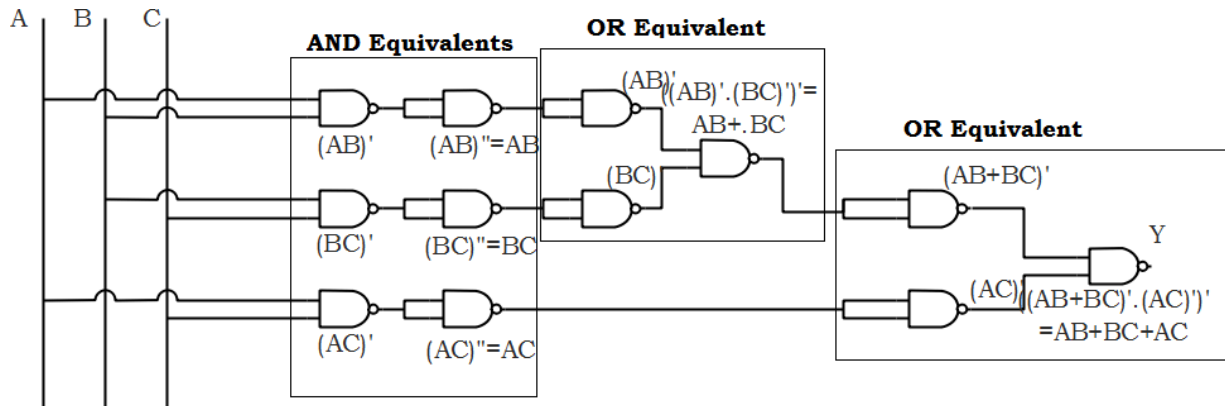


1.2.2. Realize the following Boolean expression using only NAND gates.

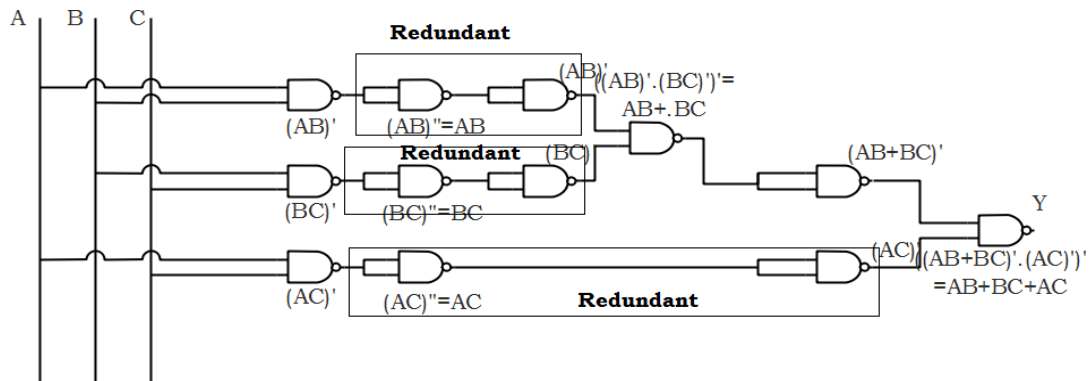
$$Y = AB + BC + AC$$

Logic diagram

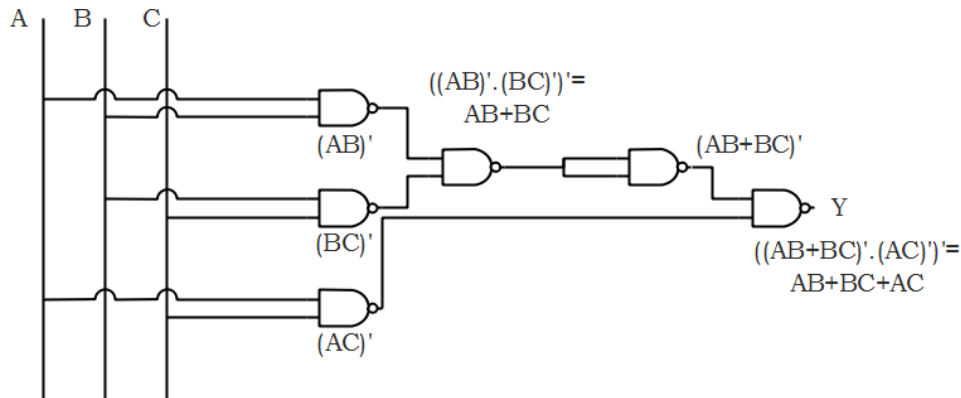
Step-1: Replace basic gates by NAND equivalents



Step-2: Eliminate two single input NAND gates are connected in series.



Step-3: Draw the resultant logic circuit.



1.3. Representation of Boolean Expressions

The relationship between Boolean variables and output variable is called Boolean expression, the Boolean expressions can be represented in two different forms, they are,

1. Sum of Products (SOP) form and
2. Product of Sums (POS) form

1.3.1. Sum of Product (SOP) form

The Boolean Expressions in which the product of input variables are summed together for output high.

Example: Consider a truth table shown in table 1.8.

A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

Table 1.8: Truth table

The Boolean expression for Y is

$$Y = \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C} + A\bar{B}C + ABC \text{ --- (1)}$$

NOTE: $A = 0$ is represented as $\bar{A}$ and $A = 1$ will be represented as A

Expression (1) is a standard or canonical sum of product form, which is directly derived from the truth table.

Sum variable can be minimized using Boolean algebra rules.

$$Y = \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C} + AC(\bar{B} + B)$$

$$Y = \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C} + AC$$

$$Y = \bar{A}\bar{B}C + \bar{A}B\bar{C} + A(\bar{B}\bar{C} + C)$$

$$Y = \bar{A}\bar{B}C + \bar{A}B\bar{C} + A(\bar{B} + C)$$

$$\begin{aligned}
 Y &= \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B} + AC \\
 Y &= \bar{A}\bar{B}C + A\bar{B} + \bar{A}B\bar{C} + AC \\
 Y &= (\bar{A}C + A)\bar{B} + \bar{A}B\bar{C} + AC \\
 Y &= (C + A)\bar{B} + \bar{A}B\bar{C} + AC \\
 Y &= A\bar{B} + C\bar{B} + \bar{A}B\bar{C} + AC \text{ --- (2)}
 \end{aligned}$$

Expression (2) is the simplified form of canonical SOP form called, **minimal SOP form**.

NOTE:

1. Canonical SOP form to minimal SOP form and vice versa can also be derived using truth table.
2. Each product terms of SOP form is called **minterms**.
3. Canonical SOP form of Boolean expressions can also be written using decimal equivalent of input variables for the output high.

Example: for the Boolean expression (1), the output is high for ABC=001, ABC=010, ABC=100, ABC=101 and ABC=111.

The decimal equivalent of ABC=001 is '1', ABC=010 is '2', ABC=100 is '4', ABC=101 is '5' and ABC=111 is '7'

Therefore, Y can also be expressed as

$$\begin{aligned}
 Y(A, B, C) &= (m_1, m_2, m_4, m_5, m_7) \\
 &\text{OR} \\
 Y(A, B, C) &= \sum m(1, 2, 4, 5, 7)
 \end{aligned}$$

Problem: Refer the truth table shown in table 1.9., write the Boolean expression in canonical SOP form and minimal SOP form. Also write the different ways of writing canonical SOP form.

A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Standard or canonical form representation

$$\begin{aligned}
 Y &= \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}C + AB\bar{C} + ABC \\
 \text{other representations OR} \quad &Y(A, B, C) \\
 &= (m_1, m_2, \\
 &\quad m_5, m_6, \\
 &\quad m_7) \\
 &Y(A, B, C) \\
 &= \sum \\
 &m((1, 2, 5, 6, \\
 &\quad 7)
 \end{aligned}$$

Simplification of Boolean expression using Boolean algebra rules

$$Y = \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}C + AB\bar{C} + ABC$$

$$Y = \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}C + AB(\bar{C} + C)$$

$$Y = \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}C + AB$$

$$Y = \bar{A}\bar{B}C + A\bar{B}C + \bar{A}B\bar{C} + AB$$

$$Y = (\bar{A} + A)\bar{B}C + \bar{A}B\bar{C} + AB$$

$$\begin{aligned}
 Y &= \bar{B}C + \bar{A}B\bar{C} + AB \\
 Y &= \bar{B}C + (\bar{A}C + A)B \\
 Y &= \bar{B}C + (\bar{C} + A)B \\
 Y &= \bar{B}C + B\bar{C} + AB - \text{minimal SOP form}
 \end{aligned}$$

1.3.2. Product of Sum (POS) form

The Boolean Expressions in which the Sum of input variables are multiplied together for output low.

Example: Consider a truth table shown in table 1.9.

A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

Table 1.9: Truth table

The Boolean expression for Y is

$$Y = (A + B + C)(A + \bar{B} + \bar{C})(\bar{A} + \bar{B} + C) \text{ --- (1)}$$

NOTE: A = 0 is represented as $\bar{A}$ and A = 1 will be represented as $\bar{\bar{A}}$

Expression (1) is a standard or canonical Products of sum form, which is directly derived from the truth table.

Sum variable can be minimized using Boolean algebra rules.

$$\begin{aligned}
 Y &= (A + B + C)(A + \bar{B} + \bar{C})(\bar{A} + \bar{B} + C) \\
 Y &= (A + (B + C)(\bar{B} + \bar{C}))(\bar{A} + \bar{B} + C) \quad (\because (A + X)(A + Y) = A + XY) \text{ Distributive law} \\
 Y &= (A + C + B\bar{B})(\bar{A} + \bar{B} + C) \\
 Y &= (A + C)(\bar{A} + \bar{B} + C) \\
 Y &= (C + A)(\bar{A} + \bar{B}) \text{ --- (2)}
 \end{aligned}$$

Expression (2) is the simplified form of canonical POS form called, **minimal POS form**.

NOTE:

1. Canonical POS form to minimal POS form and vice versa can also be derived using truth table.
2. Each Sum terms of POS form is called **maxterm**.
3. Canonical POS form of Boolean expressions can also be written using decimal equivalent of input variables for the output high.

Example: for the Boolean expression (1), the output is low for ABC=000, ABC=011 and ABC=110.

The decimal equivalent of ABC=000 is '0', ABC=011 is '3', and ABC=110 is '6'
Therefore, Y can also be expressed as

OR

$$Y(A, B, C) = (M_0, M_3, M_6)$$

$$Y(A, B, C) = \sum M(0,3,6)$$

Problem: Refer the truth table shown in table 1.9., write the Boolean expression in canonical POS form and minimal POS form. Also write the different ways of writing canonical SOP form.

A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Standard or canonical form representation

$$Y = (A + B + C)(A + \bar{B} + \bar{C})(\bar{A} + B + C)$$

other representations OR

$$Y(A, B, C) = (M_0, M_3, M_4)$$

$$Y(A, B, C) = \sum M(0,3,4)$$

Simplification of Boolean expression using Boolean algebra rules

$$Y = (A + B + C)(A + \bar{B} + \bar{C})(\bar{A} + B + C)$$

$$Y = (A + B + C)(\bar{A} + B + C)(A + \bar{B} + \bar{C})$$

$$Y = (A\bar{A} + B + C)(A + \bar{B} + \bar{C})$$

$$Y = (B + C)(A + \bar{B} + \bar{C}) \text{ --- Minimal POS form}$$

1.4. K-Map simplification (Karnaugh Map simplification)

The simplification of Boolean expressions using Boolean algebraic rules is not unique and most of the cases, the resultant expression is not in minimal form. In order to get the uniqueness and final minimal form, K-map technique will be used. In the following section, the introduction to K-maps, grouping of variables and simplification procedures are discussed with examples.

NOTE:

Number of cells in K-map = number of possible cases

No. of possible cases = 2^N

N is the number of input variables.

Example: Number of input variables=2, then the number of cells in K-map is 4.

NOTE: K-maps can take wither POS form or SOP form, in SOP form 1's are need to be grouped and in POS form o's are need to be grouped.

NOTE: Only adjacent cells will be considered for grouping, diagonal cells should not be grouped.

NOTE: grouping can be done using 2 variables, 4 variables, 8 variables, 16 variables etc., highest priority for grouping maximum variables in the above denomination. Variables are 0's for POS form and 1's for SOP form.

Procedure:

1. Select the number of cells according to the number of input variables.
2. Identify whether the given problem is SOP or POS form, minterms for SOP form and maxterms for POS form.

NOTE: In SOP form, fill the cells by 1's at corresponding minterms and otherwise fill with 0's.

NOTE: In POS form, fill the cells by 0's at corresponding maxterms and otherwise fill with 1's.

NOTE: In POS form, take the complement of the output variable to get the resultant expression.

3. group the terms in the form of rectangular, the total number of terms is 2, 4, 8, etc., try to cover as many elements as you can in one group.
4. from the groups, find the Product terms for SOP form and sum terms for POS form.

Example:

Simplify the following canonical SOP form of Boolean expression using K-map technique.

$$Y(A, B, C, D) = \sum m(0,2,3,4,6,9,11,13,15)$$

CD \ AB	00	01	11	10
00	1 0	1	1 3	1 2
01	1 4	5	7	1 6
11	12	1 13	1 15	14
10	8	1 9	1 11	10

Simplified Boolean Expression $Y = \bar{A}\bar{D} + \bar{A}\bar{B}C + AD$

UNIT-IV : Counters design and state machines

Counters

Registers can also be used to generate the specified sequence, counting the binary values either incrementally or in decrement is an example of sequence. Hence, counter is a sequential circuit, which counts the pulses either in ascending or descending order.

n-bit counter will be designed using n-flip-flops.

Counters are classified into two types based on the application of clock signal.

1. Synchronous counters and
2. Asynchronous counters.
- 3.

The clock signal is applied to all the flip-flops of a counter circuit simultaneously is called synchronous counters.

Figure (49) shows the two-bit counter, the clock signal for the two flip-flops are supplied simultaneously. $\overline{CLR}$ is an asynchronous input, logic-0 to this pin sets the initial value of all the flip-flop to zero.

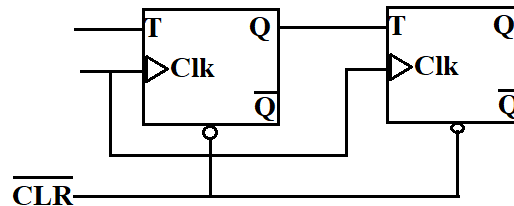


Figure 49: Example for synchronous counter

The output of the previous state flip serves as a clock input to the next state flip-flop is called asynchronous counters.

Figure (50) shows that, the clock input for the second flip-flop will be supplied from the output of first flip-flop.

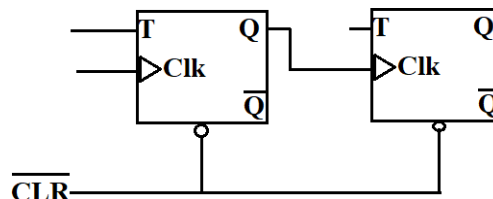


Figure 50: Example for asynchronous counter

Similarly, counters can be designed using any type of flip-flops, depending on the counting sequence, counters are further classified into three types.

4. Up-counter and
Counts the clock pulse incrementally.
5. Down-counter
Counts the clock pulse in decrement order.
6. Ring counter
Outputs the specified sequence in circular/ring format.

Asynchronous UP-Counter (Binary Ripple counter)

Counters whose counting sequence corresponds to that of binary numbers are called binary counters. The modulus of binary counter is 2^n , where n is the number of flip-flops required to design the counter.

Example:

if $n=3$, the counter performs counting from 000 to 111 (8 combinations(0 to $2^n - 1$)), hence three flip-flops are required.

Similarly

If $n=4$, the counter performs counting from 0000 to 1111 (16 combinations(0 to $2^n - 1$)), hence four flip-flops are required.

Figure (51) shows the logic diagram of 4-bit up counter using positive edge triggered T flip-flops. Each positive transition of clock makes the flip-flop to toggle.

Logic Diagram

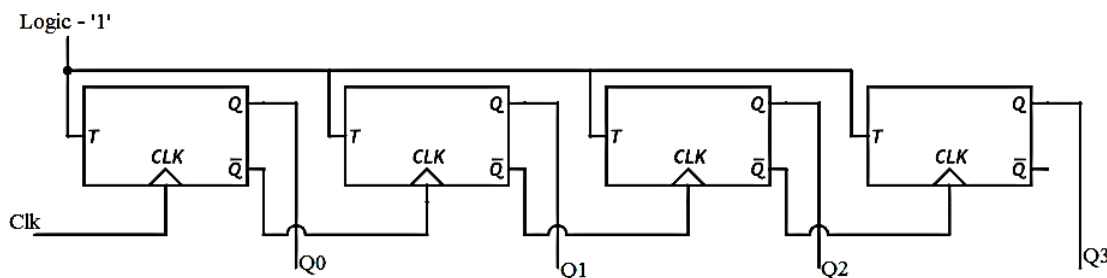


Figure 51: Example for asynchronous counter

Output Q of previous order flip-flop serves clock signal for the next order flip-flops, and clock pulse applied to all the flip-flops is not simultaneous, hence called as asynchronous counter. T inputs of all flip-flops are connected to logic-1, which acts as a toggle device. Output of the last stage flip-flop is MSB and the output of first stage flip-flop is LSB.

For every rising edge of the clock pulse the content of first stage(Q0) T-flip-flop will toggles, for every falling edge of the Q0 the content of second stage(Q1) T-flip-flop will toggles, this process continues until the last stage flip-flop.

Assume, initially, the contents of all the four T flip-flops are zero. At the rising edge of the clock pulse Q0 becomes '1' and the remaining flip-flop outputs remains zero. The later stage flip-flops

output change occurs at the next falling edge of the previous stage flip-flop outputs, shown in the timing diagram figure (52).

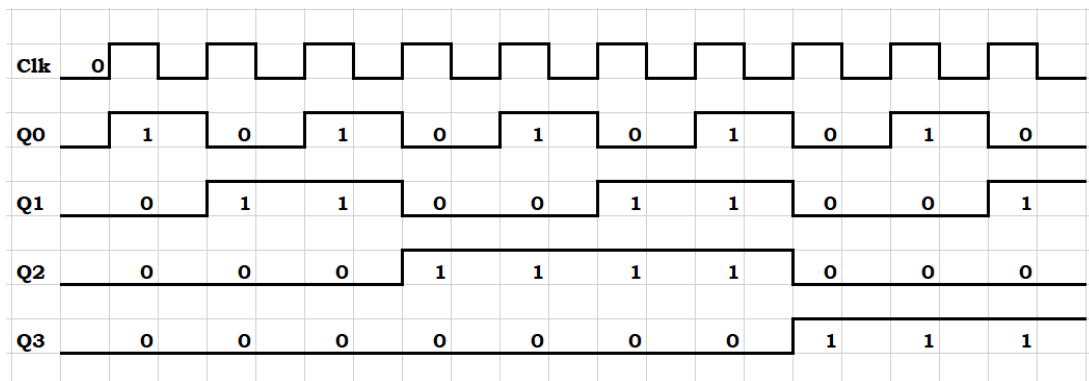


Figure 52: Timing diagram of 4-bit asynchronous binary ripple up counter

Change in the state of flip-flops occurs through the outputs of the previous stage flip-flops, that is the effect of count pulse must ripple through the counter. Hence the name ripple counter.

Asynchronous Down-Counter (Binary Ripple counter)

Figure (53) shows the logic diagram of 4-bit down counter using positive edge triggered T flip-flops. Each positive transition of clock makes the flip-flop to toggle.

Logic Diagram

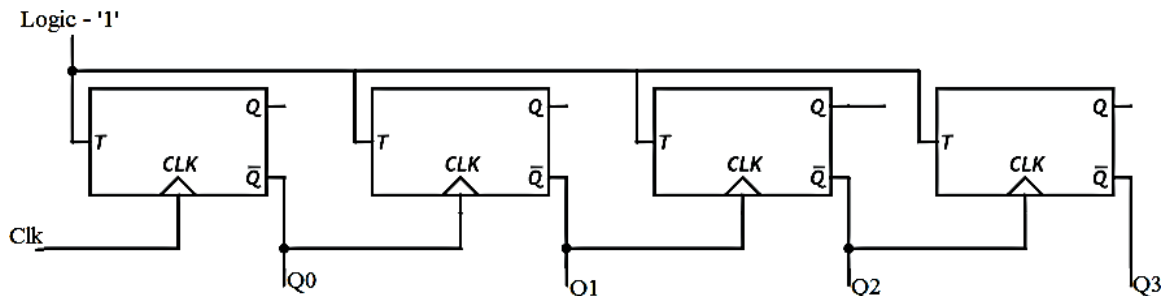


Figure 53: Timing diagram of 4-bit asynchronous binary ripple up counter

Output Q of previous order flip-flop serves clock signal for the next order flip-flops, and clock pulse applied to all the flip-flops is not simultaneous, hence called as asynchronous counter. T inputs of all flip-flops are connected to logic-1, which acts as a toggle device. Output $\bar{Q}_3$ of the last stage flip-flop is MSB and the output $\bar{Q}_0$ of first stage flip-flop is LSB.

For every rising edge of the clock pulse the content of first stage($\bar{Q}_0$) T-flip-flop will toggles, for every falling edge of the Q_0 the content of second stage(Q_1) T-flip-flop will toggles, this process continues until the last stage flip-flop.

Assume, initially, the contents of all the four T flip-flops are zero, hence complement of the outputs is 1's. At the rising edge of the clock pulse $\bar{Q}_0$ becomes '0' and the remaining flip-flop complement

outputs remains 1. The later stage flip-flops output change occurs at the next falling edge of the previous stage flip-flop outputs, shown in the timing diagram figure (54).

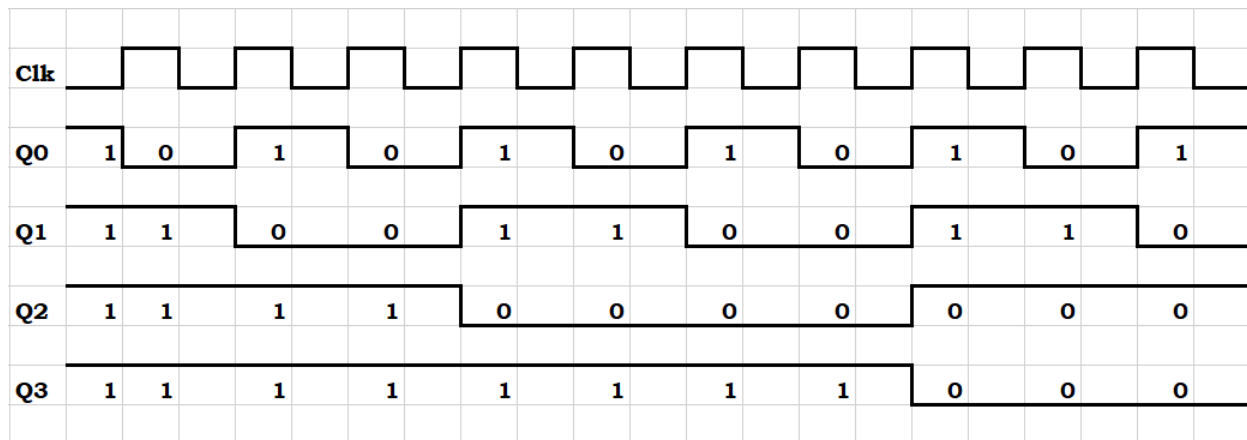


Figure 54: Timing diagram of 4-bit asynchronous binary ripple up counter

Change in the state of flip-flops occurs through the outputs of the previous stage flip-flops, that is the effect of count pulse must ripple through the counter. Hence the name ripple counter

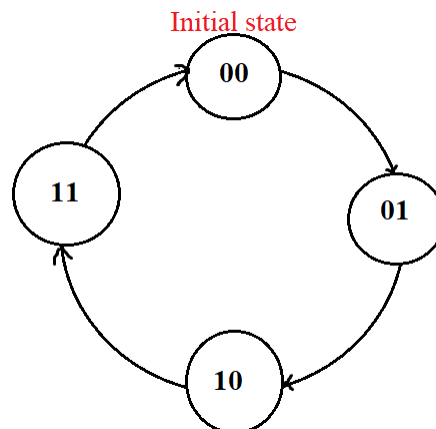
State diagram.

A graphical representation of the behavior of counters or any sequential circuits is called state diagram. State diagrams helps for the design of counters easily, which shows the transition from present states to the next states.

Procedure to draw the state diagram.

- Each state is represented by a circle, and the present state should be mentioned inside the circle.
- Use arrow associated lines to show the transition from present state to the next state.
NOTE: if the present state and next state is same, then connect the arrow associated line to the same state.

Example: Two bit up-counter, four states counting from 00 to 11.



State table

The information contained in the state diagram is tabulated in a tabular form is called state table also called state synthesis table.

Example: Two-bit counter

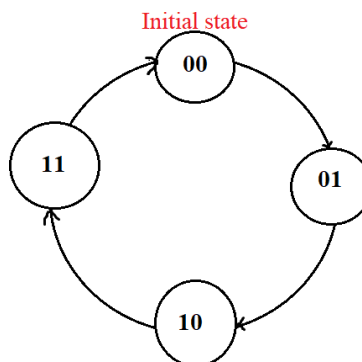
Present states		Next states	
Q_1	Q_0	Q_1^+	Q_0^+
0	0	0	1
0	1	1	0
1	0	1	1
1	1	0	0

Procedure to design synchronous counters.

1. Identify the number of flip-flops required based on the number of bits of sequence.
2. Draw the state diagram
3. Write the excitation table of the chosen flip-flop.
4. Obtain the excitation table for the complete circuit.
5. simplify the excitation equations using K-map.
6. Draw the logic diagram.
7. Verify the logic diagram by function table and timing diagram.

In the following section, the design of two-bit synchronous counter is discussed using T flip-flop.

1. No. of flip-flops is required =2, two-bit counter – sequence is 00, 01, 10 and 11.
2. State diagram



3. Excitation table of T-flip-flop

States		Excitations
Q	Q^+	T
0	0	0
0	1	1

1	0	1
1	1	0

4. Excitation table for complete circuit

Present states		Next states		Excitations	
Q_1	Q_0	Q_1^+	Q_0^+	T_1	T_0
0	0	0	1	0	1
0	1	1	0	1	1
1	0	1	1	0	1
1	1	0	0	1	1

5. K-map simplification

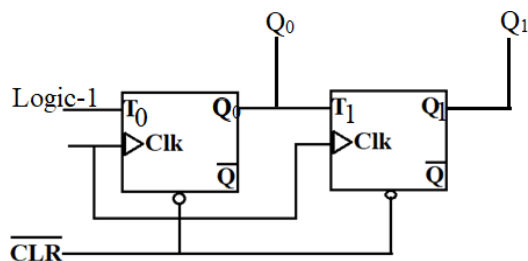
		T_1	
		Q_0	
Q_1	0	0	1
	1	0	1

$$T_1 = Q_0$$

		T_0	
		Q_0	
Q_1	0	1	1
	1	1	1

$$T_0 = 1$$

6. Logic diagram



7. Verification
Function table

Clock Pulse	Outputs	
	Q_1	Q_0
0	0	0
1	0	1
2	1	0
3	1	1
4	0	0

Timing diagram

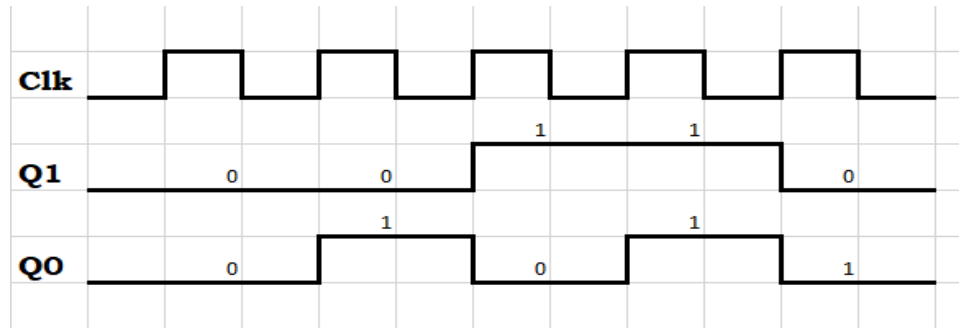


Figure: Timing diagram of two-bit counter

Similarly, counters can be designed using any type of flip-flops, depending on the counting sequence, counters are further classified into three types.

7. Up-counter and

Counts the clock pulse incrementally.

8. Down-counter

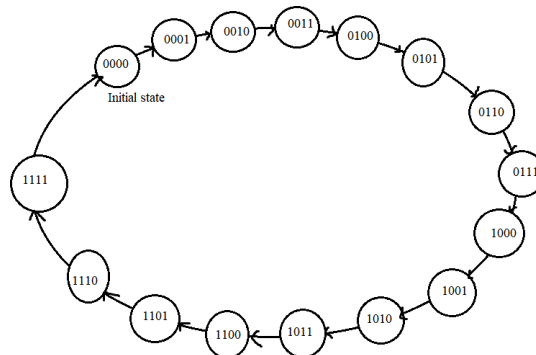
Counts the clock pulse in decrement order.

9. Ring counter

Outputs the specified sequence in circular/ring format.

1. 4-bit synchronous up-counter using T flip-flops.

1. No. of Flip-flops required=04.
2. State diagram



3. Excitation table of T-flip-flop

States		Excitations
Q	Q ⁺	T
0	0	0
0	1	1
1	0	1
1	1	0

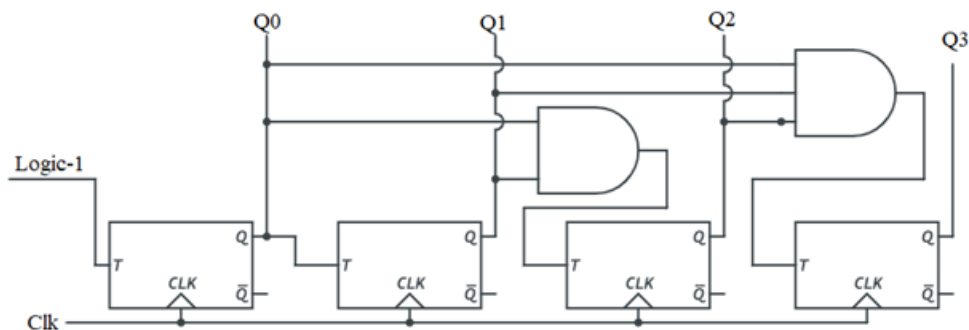
4. Excitation table of complete circuit

Present states				Next states				Excitations			
Q3	Q2	Q1	Q0	Q3+	Q2+	Q1+	Q0+	T3	T2	T1	T0
0	0	0	0	0	0	0	1	0	0	0	1
0	0	0	1	0	0	1	0	0	0	1	1
0	0	1	0	0	0	1	1	0	0	0	1
0	0	1	1	0	1	0	0	0	1	1	1
0	1	0	0	0	1	0	1	0	0	0	1
0	1	0	1	0	1	1	0	0	0	1	1
0	1	1	0	0	1	1	1	0	0	0	1
0	1	1	1	1	0	0	0	1	1	1	1
1	0	0	0	1	0	0	1	0	0	0	1
1	0	0	1	1	0	1	0	0	0	1	1
1	0	1	0	1	0	1	1	0	0	0	1
1	0	1	1	1	1	0	0	0	1	1	1
1	1	0	0	1	1	0	1	0	0	0	1
1	1	0	1	1	1	1	0	0	0	1	1
1	1	1	0	1	1	1	1	0	0	0	1
1	1	1	1	0	0	0	0	1	1	1	1

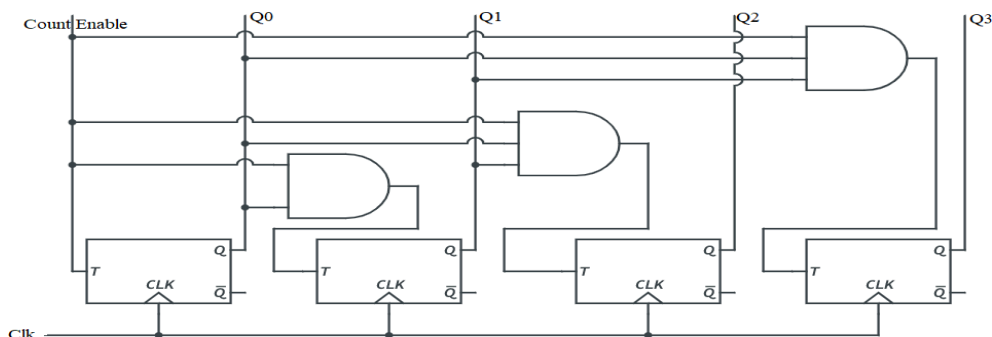
5. K-map Simplification

$$T_3 = Q_2 Q_1 Q_0, T_2 = Q_1 Q_0, T_1 = Q_0, T_0 = 1$$

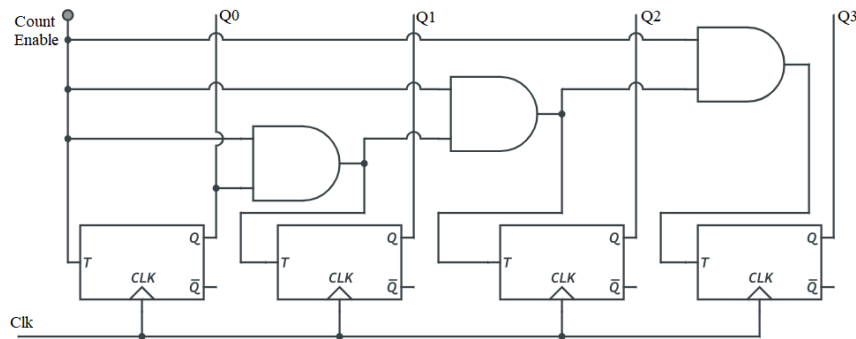
6. Logic Diagram



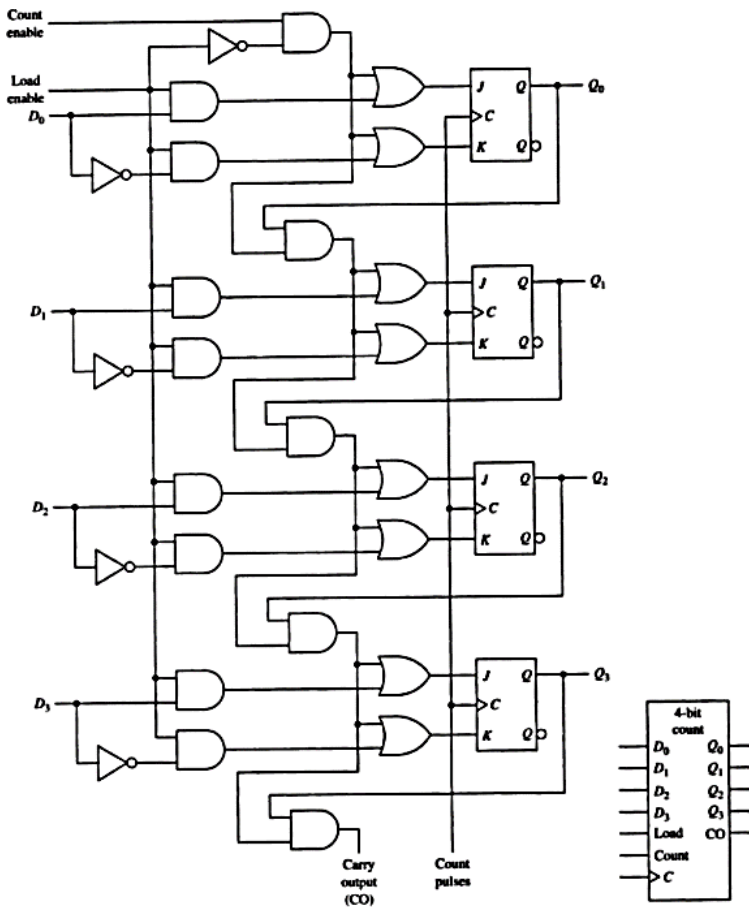
a. NOTE: 4-bit synchronous counter with count enable input



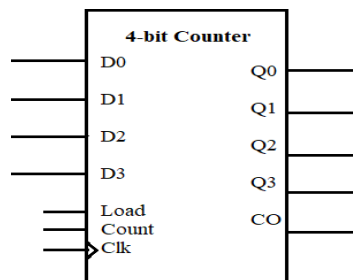
b. NOTE: 4-bit synchronous counter variation (design using only two input AND gates)



c. 4-bit synchronous counter with parallel loading using JK flip-flops

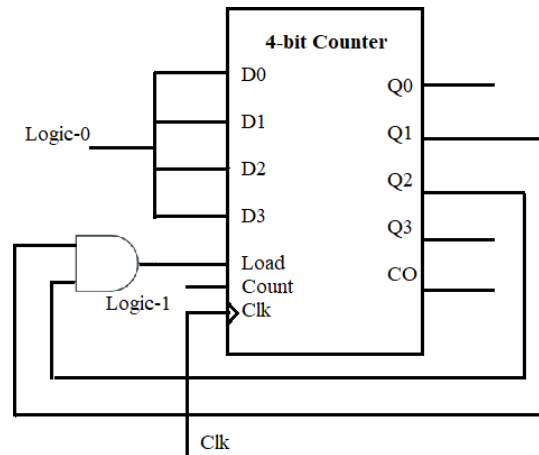


Logic Symbol

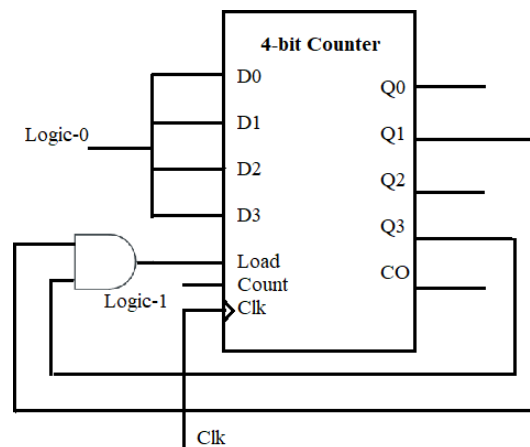


2. MOD counters with parallel loading

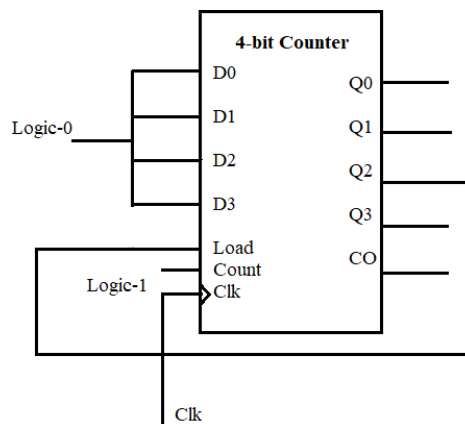
Example-1: Synchronous Mod-6 counter with parallel loading



Example-2: MOD-10 Counter



Example-3: MOD-4 Counter



3. Realization of 8-bit synchronous counter using two 4-bit counters

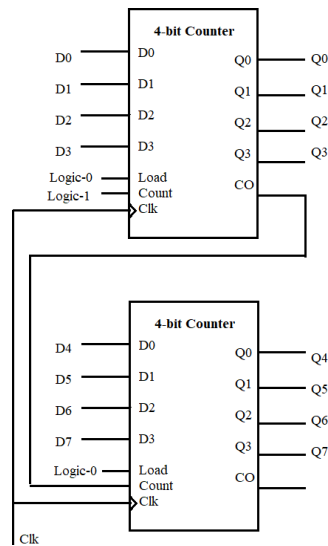


Figure : 8-bit synchronous counter

Counters based on shift registers

registers can also be used to generate the sequences repeatedly and acts as the counters, there are two types of counters Shift based on shift registers.

1. Ring Counter

The mod of ring counter is n , where n is the number of flip-flops are used in the design.

In the following section discusses the 4-bit ring counter using four D flip-flops shown in figure (). The previous stage outputs of flip-flops serve as the input to next stage flip-flops and last stage flip-flop output serves as the input of first stage flip-flop, which shifts the contents of flip-flops position wise and also in circular.

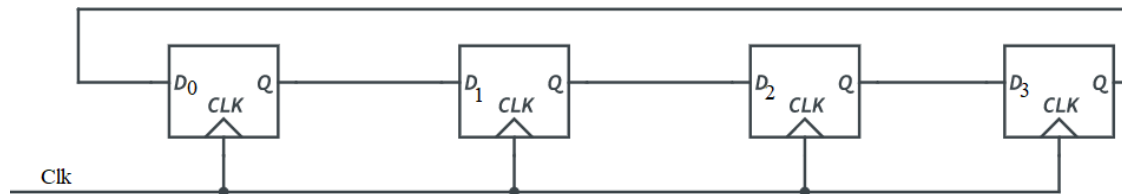


Figure: 4-bit ring counter

Truth table

Clk	Q0	Q1	Q2	Q3
0	1	0	0	0
1	0	1	0	0
2	0	0	1	0
3	0	0	0	1
4	1	0	0	0

Initially, the contents of flip-flops ($Q_0Q_1Q_2Q_3$) are assumed as (1000), at every rising edge of the clock pulse, the content of previous stage flip-flop will be shifted to the next stage flip-flop. The ring counter generates the four sequence and the sequence repeats after every four clock pulses, hence the mod of this counter is four.

Timing diagram

Figure shows the timing diagram of 4-bit ring counter.

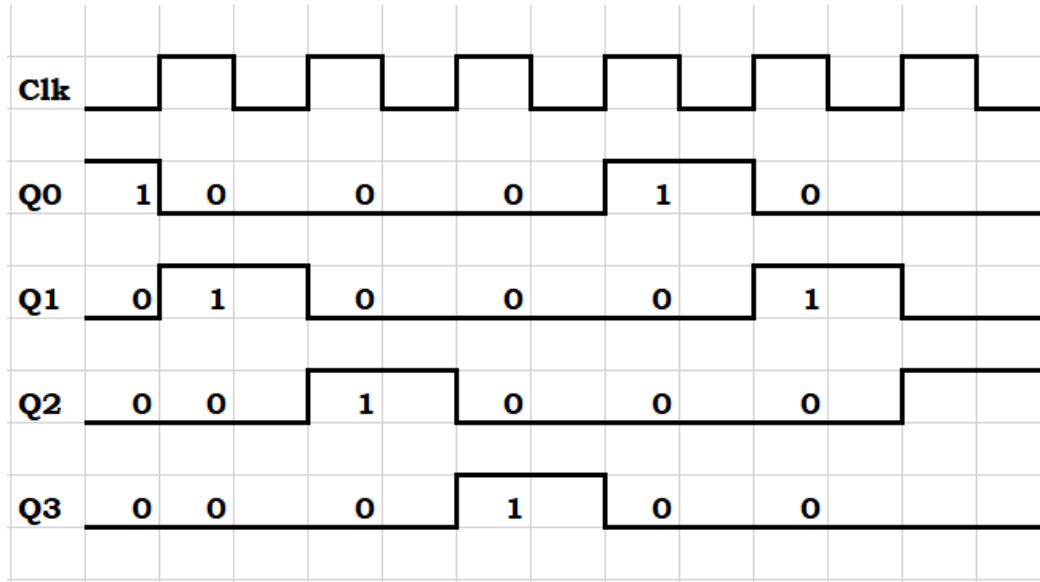


Figure: Timing diagram of 4-bit ring counter.

2. Twisted ring counter

The twisted ring counter is the modification of ring counter; complemented output of last stage serves as the input to the first stage flip-flop. The modulus of the twisted ring counter is $2n$, where n is the number of flip-flops are used in the design. Figure () shows the logic diagram of 4-bit twisted ring counter using D flip-flops.

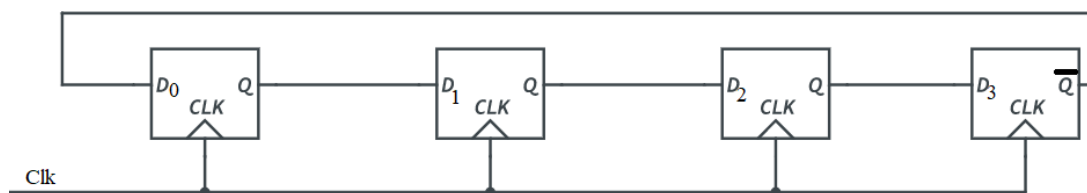


Figure : 4-bit twisted ring counter

Initially, the contents of flip-flops ($Q_0Q_1Q_2Q_3$) are assumed as (0000), at the first rising edge of the clock pulse $D_0 = \bar{Q}_3$ and hence, $Q_0 = D_0$, $Q_1 = D_1$, $Q_2 = D_2$, $Q_3 = D_3$ this data shift position wise at the flip-flops at every rising edge of the clock pulse, The twisted ring counter generates the eight sequence and the sequence repeats after every eight clock pulses, hence the mod of this counter is eight. The twisted ring counter is also called Johnson counter and switch tail counter.

Truth Table:

Clk	Q0	Q1	Q2	Q3
0	0	0	0	0
1	1	0	0	0
2	1	1	0	0
3	1	1	1	0
4	1	1	1	1
5	0	1	1	1
6	0	0	1	1
7	0	0	0	1
8	0	0	0	0

Timing diagram

Figure shows the timing diagram of Johnson counter.

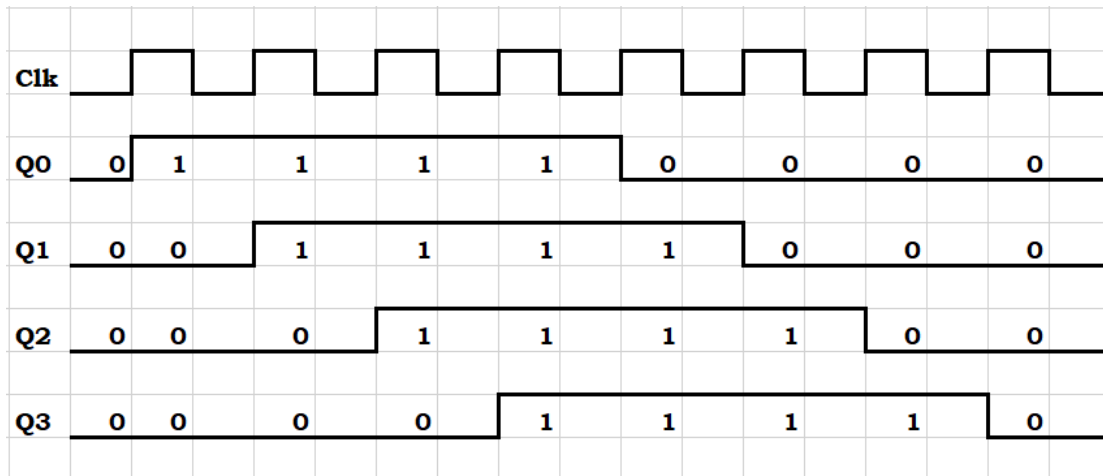


Figure : Timing diagram of twisted ring counter

NOTE: State machines, state table and state diagram concepts need to be added

